

Class Notes: Data Structures

Unit 1: Introduction to Data Structures

Course: Data Structures

Level: Undergraduate (BCA/B.Tech/B.Sc. CS)

1. Introduction

A **Data Structure** is a systematic way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

It provides a means to manage large amounts of data for efficient processing.

Definition

Data Structure: A collection of data elements organized in a specific manner along with operations that can be performed on them.

Why Data Structures?

- Efficient storage of data
- Faster searching and retrieval
- Easy insertion and deletion
- Better memory utilization
- Improves program performance

Real-Life Examples

Example	Data Structure Used
Student database	Array / Linked List
Browser history	Stack
Printer queue	Queue
GPS Navigation	Graph
Folder hierarchy	Tree

2. Basic Terminologies

2.1 Data

Data is a collection of raw facts and figures.

Example:

10
Ram
85
A+

These individual values are called **data**.

2.2 Information

Processed and meaningful data is called **Information**.

Example

Roll	Name	Marks
101	Ram	85

This table gives useful information.

2.3 Data Item

A single unit of data.

Example

Age = 22

22 is a data item.

2.4 Group Item

A collection of related data items.

Example

Student
Name
Age
Department

2.5 Elementary Data Organization

Elementary data organization consists of the following hierarchy:

Bit
↓
Byte
↓
Field
↓
Record
↓
File
↓
Database

Example

Field

Name

Record

Name : John
Age : 21
Dept : CSE

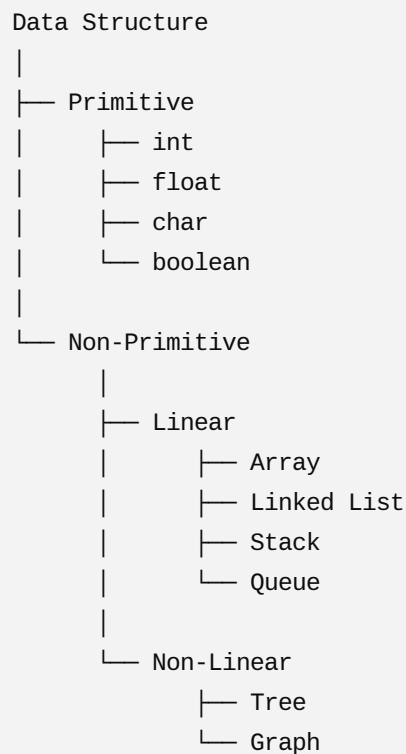
File

Collection of student records.

Database

Collection of multiple files.

3. Classification of Data Structures



4. Data Structure Operations

The common operations performed on data structures are:

(a) Traversal

Visiting every element exactly once.

Example

Array

10 20 30 40

Traversal

10 → 20 → 30 → 40

Time Complexity

O(n)

(b) Insertion

Adding a new element into a data structure.

Example

Before

10 20 40

Insert 30

After

10 20 30 40

Worst Case Complexity

Array: **$O(n)$**

(c) Deletion

Removing an element.

Example

Before

10 20 30 40

Delete 20

After

10 30 40

Worst Case

Array: **$O(n)$**

(d) Searching

Finding an element.

Example

Find 30

10 20 30 40

Output

Position = 3

(e) Sorting

Arranging data in ascending or descending order.

Example

Before

30 10 20

After

10 20 30

(f) Merging

Combining two data structures.

Example

10 20

+

30 40

↓

10 20 30 40

5. Algorithm

An **Algorithm** is a finite sequence of well-defined instructions to solve a problem.

Characteristics

- Input
 - Output
 - Definiteness
 - Finiteness
 - Effectiveness
-

Example

Find the largest number.

Algorithm

1. Start
 2. Read A and B
 3. If $A > B$
 4. Print A
 5. Else Print B
 6. Stop
-

6. Analysis of an Algorithm

Algorithm analysis determines the amount of:

- Time required
- Memory required

before implementing the program.

There are two types:

(i) Time Complexity

Measures execution time.

(ii) Space Complexity

Measures memory usage.

Cases of Analysis

Best Case

Minimum execution time.

Example

Searching first element.

```
10 20 30 40
```

```
Find 10
```

Complexity

$O(1)$

Worst Case

Maximum execution time.

Example

Searching last element.

```
10 20 30 40
```

```
Find 40
```

Complexity

$O(n)$

Average Case

Expected running time over all inputs.

7. Asymptotic Notations

Asymptotic notations describe the growth rate of algorithms for large input sizes.

(i) Big O Notation $O()$

Upper bound.

Worst-case complexity.

Example

Linear Search

$O(n)$

(ii) Big Omega $\Omega()$

Lower bound.

Best-case complexity.

Example

Searching first element

$\Omega(1)$

(iii) Big Theta $\Theta()$

Exact bound.

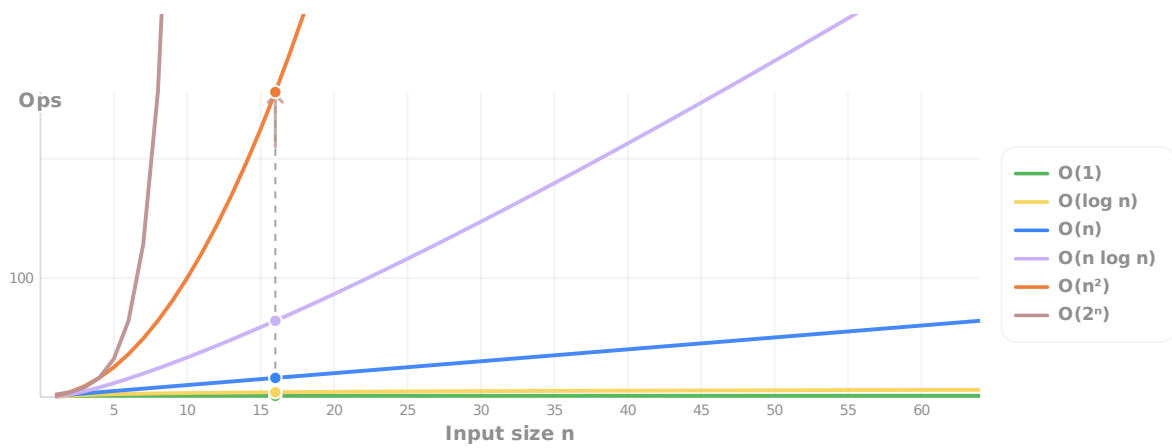
Average or tight bound.

Example

Merge Sort

$\Theta(n \log n)$

Common Complexity Classes



$$n = 16 : (1, \log_2 n, n) = (1, 4, 16)$$

$$(n \log_2 n, n^2) = (64, 256)$$

$$2^n = 65,536$$

n 4 8 **16** 32 64

Complexity	Name	Example
$O(1)$	Constant	Array indexing
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linear Logarithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(2^n)$	Exponential	Recursive Fibonacci

8. Time-Space Trade-off

Sometimes we use more memory to reduce execution time.

Or,

Use less memory but spend more execution time.

Example 1

Store calculated factorial values.

Extra Memory ✓

Less Computation ✓

Example 2

Binary Search

Requires sorted array.

Extra work for sorting initially

But searching becomes very fast.

Comparison

More Time	Less Memory
Calculate repeatedly	No extra storage

Less Time	More Memory
Store intermediate results	Extra storage

9. Searching

Searching is the process of locating an element in a collection of data.

Two popular searching techniques are:

1. Linear Search
 2. Binary Search
-

10. Linear Search

Linear Search checks every element one by one until the required element is found.

Algorithm

1. Start from first element.

2. Compare with target.
3. If matched, return position.
4. Otherwise move to next element.
5. Repeat until end.

Example

Array

10 25 15 30 40

Search = 30

Comparisons

10 ✗

25 ✗

15 ✗

30 ✓

Position = 4

Pseudocode

```
LinearSearch(A, n, key)
```

```
for i = 0 to n-1
    if A[i] == key
        return i
return -1
```

Complexity

Best Case

$O(1)$

Worst Case

$O(n)$

Average Case

$O(n)$

Advantages

- Very simple
- Works on unsorted data
- Easy implementation

Disadvantages

- Slow for large datasets
-

11. Binary Search

Binary Search repeatedly divides the search interval into two halves.

Condition: The array must be sorted.

Working

Given

```
10 20 30 40 50 60 70
```

Search = 50

Step 1

```
Middle = 40
```

$50 > 40$

Search right half.

Step 2

```
50 60 70
```

Middle = 60

$50 < 60$

Search left half.

Step 3

```
50
```

Found.

Pseudocode

```
BinarySearch(A, low, high, key)
```

```
while low <= high
```

```
    mid = (low + high)/2
```

```
    if A[mid] == key
```

```
        return mid
```

```
    else if key < A[mid]
```

```
        high = mid - 1
```

```
    else
```

```
        low = mid + 1
```

```
return -1
```

Visualisation

55 > 44

Target 55

Indices 0-9 · 10 candidates

L

H

M



Next: 5-9

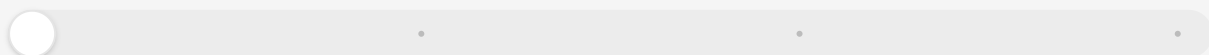
Candidates 10 → 5

Keep the right half

Exclude indices 0-4; continue with 5 candidates.

Target

55



Complexity

Best Case

$O(1)$

Worst Case

$O(\log n)$

Average Case

$O(\log n)$

Advantages

- Very fast
- Suitable for large datasets

Disadvantages

- Requires sorted data
- Slightly more complex than Linear Search

12. Comparison of Linear Search and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Sorted or Unsorted	Must be Sorted
Technique	Sequential	Divide and Conquer
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Suitable for	Small datasets	Large sorted datasets
Implementation	Easy	Moderate

13. Summary

- A **data structure** organizes data for efficient processing.

- Basic operations include **traversal, insertion, deletion, searching, sorting, and merging**.
- Algorithm analysis focuses on **time complexity** and **space complexity**.
- **Asymptotic notations** (Big O, Big Ω , and Big Θ) describe algorithm efficiency.
- **Time-space trade-off** balances execution speed with memory usage.
- **Linear Search** is simple and works on unsorted data but has **$O(n)$** time complexity.
- **Binary Search** is much faster with **$O(\log n)$** complexity but requires the data to be sorted.